



Logo / cabecera Cetmix Tower

Cetmix Tower

Guía Rápida Técnica para efactusv

Instalación → Servidor → Flight Plans → Jets → Instancia Odoo

Odoo 18.0 | Cetmix Tower 18.0.2.0.0 | fork efactusv

efactusv-cetmix-tower · rama 18.0

1. Conceptos Básicos

Cetmix Tower convierte Odoo en un panel de control de infraestructura SaaS.

Concepto	Modelo	Qué es
Server	cx.tower.server	VPS/Dedicado al que Tower se conecta por SSH.
Command	cx.tower.command	Unidad de trabajo: SSH, Python, archivo, plan, etc.
Flight Plan	cx.tower.plan	Secuencia de comandos con lógica condicional (playbook).
File Template	cx.tower.file.template	Modelo de archivo con variables {{ var }}.
Variable	cx.tower.variable	Valor con nombre. Cascada: jet → servidor → global.
Key / Secret	cx.tower.key	SSH keys y secretos. Enmascarados: #!cxtower.secret.x!#
Jet	cx.tower.jet	Instancia con ciclo de vida. 1 jet = 1 cliente Odoo.

Regla de oro: Servidor = el hierro. Jet = lo que corre y facturas a un cliente.

Arquitectura de la plataforma SaaS:



Módulos del repositorio

Módulo	Necesario?	Función
cetmix_tower_server	Obligatorio	Núcleo: servers, commands, plans, files, variables, jets, logs.
cx_web_refresh_from_backend	Obligatorio	Refresca vistas al terminar comandos largos.
cetmix_tower_yaml	Muy recomendado	Export/Import YAML. Versionado y promoción entre entornos.
cetmix_tower_server_queue	Muy recomendado	Cola asíncrona vía queue_job (OCA). Evita bloqueos HTTP.
cetmix_tower_git	Recomendado	Modela repos Git, genera repos.yaml de git-aggregator.
cetmix_tower_webhook	Opcional	Endpoints HTTP entrantes con autenticador.
cetmix_tower_ovh / _aws	Según proveedor	Conecta ovh o boto3 en comandos Python.
efactusv_tower_contract	SaaS	Botón Create Instance en contrato OCA. Automatiza ciclo de vida.

2. Instalación

2.1 Dependencias Python

```
cd /ruta/a/efactusv-cetmix-tower
pip install -r requirements.txt
# Clave: paramiko<4.0, tldextract, dnspython, ansi2html, pyyaml
```

NOTA Py3.13: Python 3.13 eliminó re.template. Este fork ya incluye el fix (commit c71707a).

2.2 Módulos OCA requeridos

Repositorio OCA	Módulo	Para
server-tools	rpc_helper	cetmix_tower_server
web	web_notify	Notificaciones en vivo
queue	queue_job	cetmix_tower_server_queue
contract	contract	efactusv_tower_contract

2.3 Orden de instalación en Odoo

Aplicaciones → filtro Apps desactivadas:

1. **cetmix_tower_server** (arrastra dependencias)
2. **cetmix_tower_yaml**
3. **cetmix_tower_server_queue** (requiere queue_job)
4. **cetmix_tower_git**, **cetmix_tower_webhook**
5. **cetmix_tower_ovh** / **_aws** (solo tu proveedor)
6. **efactusv_tower_contract** (último)

```
./odoo-bin -c odoo.conf -d efactusv_prod \
-i cetmix_tower_server,cetmix_tower_yaml,cetmix_tower_server_queue,\
cetmix_tower_git,cetmix_tower_webhook,cetmix_tower_ovh,efactusv_tower_contract \
--stop-after-init
```

2.4 odoo.conf

```
[options]
addons_path = /opt/odoo/odoo/addons,/opt/odoo/efactusv,\
              /opt/odoo/efactusv-cetmix-tower,/opt/odoo/queue,\
              /opt/odoo/server-tools,/opt/odoo/web,/opt/odoo/contract
server_wide_modules = base,web,queue_job
workers = 4          # > 0 OBLIGATORIO para queue_job
max_cron_threads = 2
limit_time_real = 1200 # docker pull puede tardar
limit_time_cpu = 600

[queue_job]
channels = root:2
```

2.5 nginx: proxy websocket

```
location /websocket {
    proxy_pass http://127.0.0.1:8072;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Sin esto, los comandos parecen colgados aunque hayan terminado.

3. Configuración Inicial

3.1 Permisos: grupos Odoo

Grupo	XML ID	Permisos
User	group_user	Ver servidores/jets asignados, ejecutar comandos nivel User, leer logs.
Manager	group_manager	Todo lo User + crear servidores, comandos, planes, variables, jet templates.
Root	group_root	Todo. Webhooks, autenticadores, secretos Root. 2–3 personas máximo.

3.2 Access Level (por registro)

Todos los objetos tienen **access_level**: 1=User, 2=Manager (default), 3=Root. El usuario tiene su nivel en su ficha. Solo ve registros ≤ a su nivel.

Error clásico: grupo Manager + Access Level User = ve menús pero comandos vacíos.

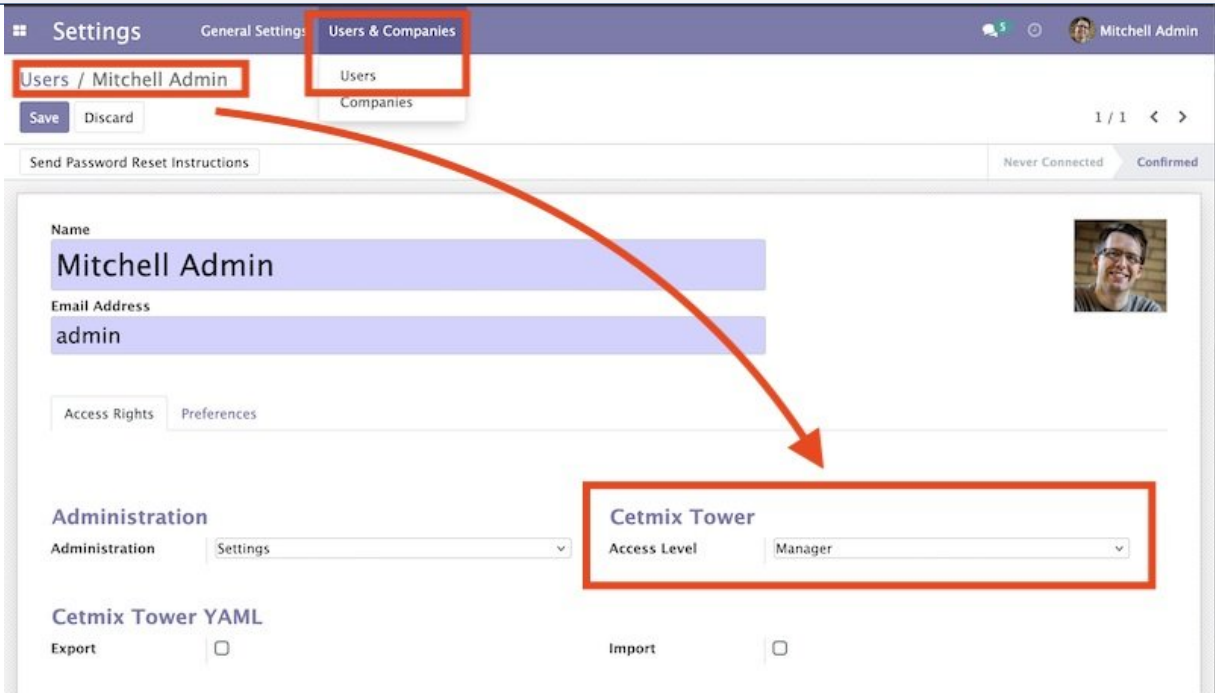


Figura 3.1 — Nivel de acceso de Tower en la ficha del usuario (Ajustes > Usuarios > Permisos de acceso > Cetmix Tower).

Plan de permisos para efactusv:

Perfil	Grupo	Access Level
Arquitecto de plataforma	Root	Root
DevOps / Soporte N2	Manager	Manager
Soporte N1	User	User
Comercial / Administración	—	—

3.3 Ajustes generales

Ajuste	Valor sugerido	Explicación
Command Timeout	900 s	Timeout SSH. -206 si se supera. Subir para imágenes grandes.
Error Notifications	Sticky	Se quedan hasta cerrar.
Success Notifications	Non-sticky	Se desvanecen solas.

3.4 Llave SSH maestra

```
ssh-keygen -t ed25519 -C "cetmix-tower@efactusv" -f ~/.ssh/tower_efactusv
# SIN passphrase: Tower no puede introducirla interactivamente.

ssh-copy-id -i ~/.ssh/tower_efactusv.pub deploy@203.0.113.10
```

En Tower: **Settings > Keys and Secrets > Nuevo** → Key Type: **SSH Key**, pegar clave privada completa. Access Level: Root.

3.5 Secretos

Sintaxis: `#!cxtower.secret.<ref>!#`. Se sustituyen antes de ejecutar, enmascarados en logs. Nunca usar `{{ }}` para contraseñas.

Referencia	Nivel	Contenido
db_password	Root	Contraseña PostgreSQL instancias
admin_passwd	Root	Contraseña maestra Odoo instancias
ovh_application_key / _secret / consumer_key	Root	Credenciales API OVHcloud
ghcr_token	Root	Token lectura registry contenedores

3.6 Variables

Cascada de resolución: **Valor en JET** → **Valor en SERVIDOR** → **Valor GLOBAL** → **(vacío/error)**. Un mismo Flight Plan sirve para 100 clientes.

Run command

Run Flight Plan

Test Connection

Command Logs

Flight Plan Logs

0 Files

Such Much Server

Running

Production

Custom

Tags

Reference

such_much_server

URL

https://memes.example.com

General Settings

Configuration

Access

Secrets

Server Logs

Git Projects

Variable	Value	Option	Access Level
Odoo Demo Version	18.0	18.0	Manager
Branch	main		Manager
Language	it	Italian	Manager
Directory	/home/{{ tower.server.ssh_username }}		Manager
Add a line			

Figura 3.2 — Pestaña Configuration de un servidor con variables sobrescritas.

Referencia	Tipo	Ejemplo	Uso
instance_name	String	cliente1	ID técnico: rutas, contenedores, BD. Sin espacios, minúsculas.
instance_domain	String	cliente1.efactusv.com	Subdominio público.
odoo_image	String	ghcr.io/porti1876/efactusv-odoo:18.0	Imagen Docker.
odoo_http_port	String	10069	Puerto local único por instancia.
odoo_gevent_port	String	10072	Puerto websocket único.
odoo_workers	Options	2 / 4 / 8	Según plan contratado.
base_dir	String	/opt/efactusv	Raíz despliegue (global).
admin_email	String	admin@efactusv.com	Para Let's Encrypt.

Variables de sistema (solo lectura, prefijo tower.):

Variable	Valor
{{ tower.server.name / .reference / .ipv4 / .status }}	Datos del servidor en contexto
{{ tower.jet.name / .reference / .state }}	Datos del jet en contexto
{{ tower.tools.uuid / .today / .now }}	UUID aleatorio, fecha/hora actuales

4. Primer Servidor

4.1 Preparar el VPS

Requisitos mínimos del anfitrión:

- Debian 12 o Ubuntu 24.04 — 2 vCPU / 4 GB cada 2–3 instancias — SSD 80 GB+
- Usuario no-root (deploy) con sudo NOPASSWD
- Docker Engine + compose; usuario en grupo docker
- nginx + certbot en el host (no en contenedor)
- Cortafuegos: 22, 80, 443. Puertos instancias solo en 127.0.0.1

```
# --- como root, una sola vez por servidor ---
adduser --disabled-password --gecos "" deploy
usermod -aG sudo deploy
echo "deploy ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deploy

# Docker
curl -fsSL https://get.docker.com | sh
usermod -aG docker deploy

# nginx y certbot
apt-get update && apt-get install -y nginx certbot python3-certbot-nginx
mkdir -p /var/www/certbot /opt/efactusv/backups
chown -R deploy:deploy /opt/efactusv

# Autorizar clave pública de Tower
mkdir -p /home/deploy/.ssh
echo 'ssh-ed25519 AAAA... cetmix-tower@efactusv' \
  >> /home/deploy/.ssh/authorized_keys
chmod 700 /home/deploy/.ssh && chmod 600 /home/deploy/.ssh/authorized_keys
```

4.2 Alta en Tower

Campo	Oblig.	Valor
Name	Sí	PROD-SV-01
Reference	Auto	Dejar vacío. No cambiar después.
Operating System	Rec.	Debian 12 / Ubuntu 24.04
Tags	Rec.	producción, región, proveedor
IPv4 / IPv6	Una	IP pública del VPS
SSH port / Username	Sí	22 / deploy
SSH Auth Mode	Sí	Key
SSH Private Key	Sí	Llave de 3.4
Use sudo	No	Without password (con NOPASSWD)

Cetmix Tower

ServersCommandsFilesToolsSettings

Servers

Create

Demo Server #1

demo1.example.com

Stopped

Operating System: Debian 10

IPv4 Address: localhost

Custom

Demo Server #2

https://dogememe.example.com

Running

Operating System: Ubuntu 20.04

IPv4 Address: localhost

Production

Figura 4.1 — Vista kanban de servidores con estado y etiquetas.

4.3 Host key y prueba de conexión

- 1. Guardar servidor → acción **Show host key** (lee y guarda la huella).
- 2. Pulsar **Test Connection**: prueba SSH + SFTP en secuencia.

Síntoma	Causa
Host key not found	Falta Show host key
Authentication failed	Clave pública no en authorized_keys o modo Auth incorrecto
Fallo SFTP	Sin permiso escritura en \$HOME o sin subsistema SFTP
Expira	Cortafuegos o puerto SSH != 22
Permission denied (sudo)	NOPASSWD no configurado

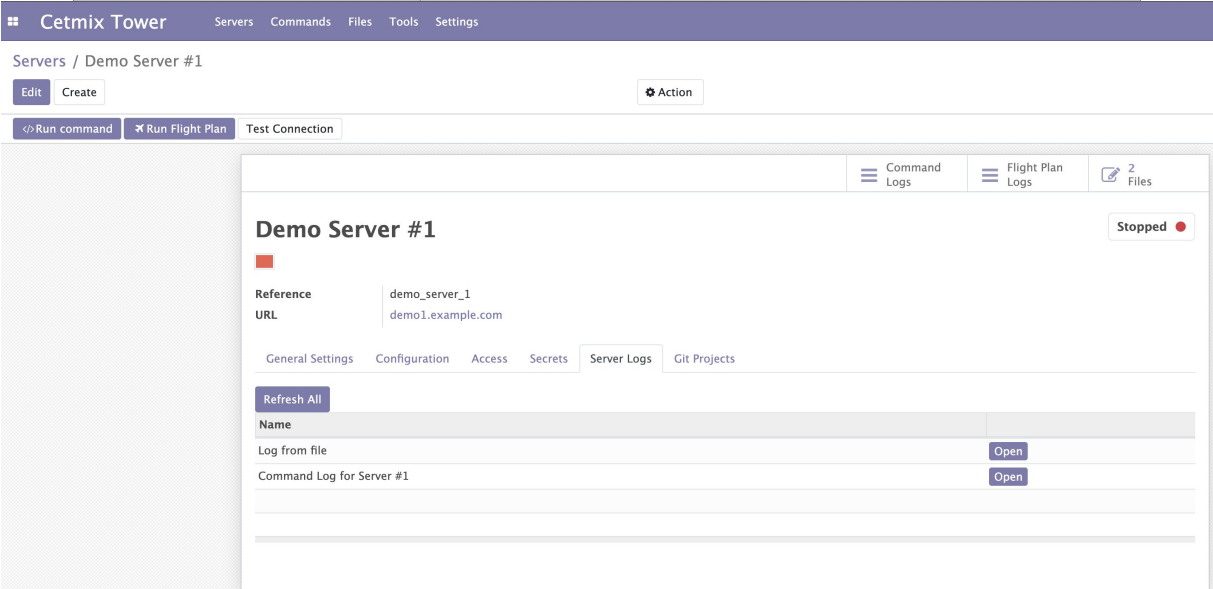


Figura 4.2 — Pestaña Server Logs con ventanas de observación configuradas.

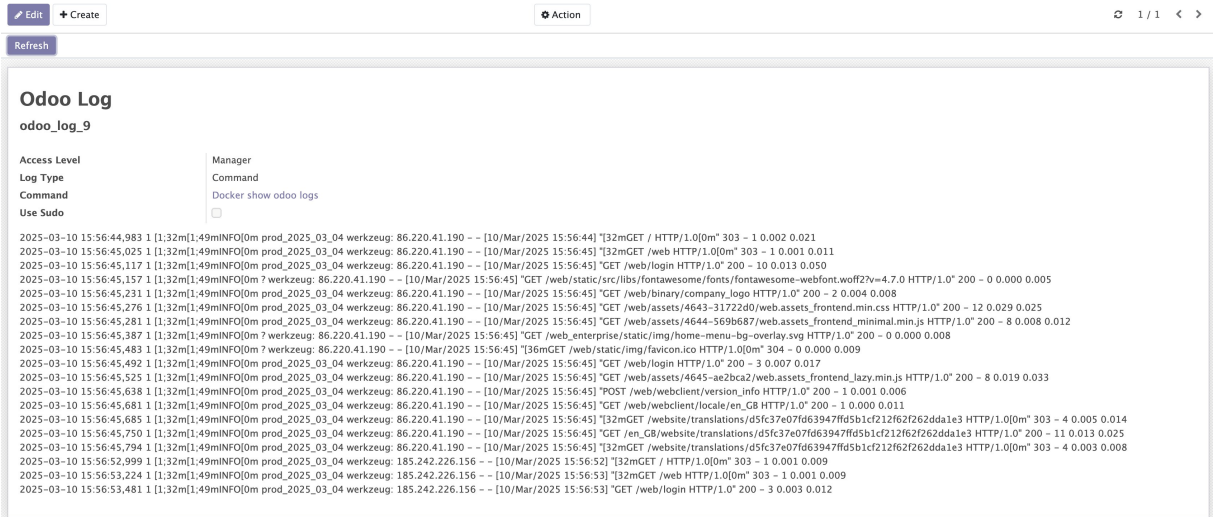


Figura 4.3 — Ventana de log tipo Command mostrando docker logs. Los colores ANSI se conservan.

Truco de soporte: crea en cada jet un Server Log tipo Command con docker logs --tail 200 odoo-{{ instance_name }} y nivel User. Soporte N1 diagnostica sin SSH.

5. Comandos

5.1 Tipos de acción

Acción	Dónde corre	Descripción
SSH Command	Servidor remoto	Ejecuta Code en shell. Exit code = resultado.
Python Code	Server Tower (Odoo)	Python dentro de Odoo. APIs, BD, decisiones.
Create/Update File	Sube al servidor	Renderiza plantilla con variables y la deposita vía SFTP.
Run Flight Plan	Orquestación	Lanza otro plan. Permite componer procesos.
Trigger Jet Action	Orquestación	Provoca transición de estado en un jet.
Create Waypoint	Orquestación	Snapshot del jet (pre-update).

Python Code NO se ejecuta en el servidor remoto. Corre en Odoo. Para Python en el server, usar SSH Command invocando python3.

5.2 Campos clave

Campo	Uso
Name / Reference	Identificación visible y técnica estable.
Access Level	Quién puede ver/ejecutar. Destructivos → Manager o Root.
Default Path	Directorio trabajo. Admite vars: {{ base_dir }}/{{ instance_name }}
Servers	Restringe servidores. Vacío = todos.
Operating Systems	Restringe SO. Ejecutar en otro → -207.
Allow Parallel Run	Desmarcado = rechaza 2ª ejecución con -201.

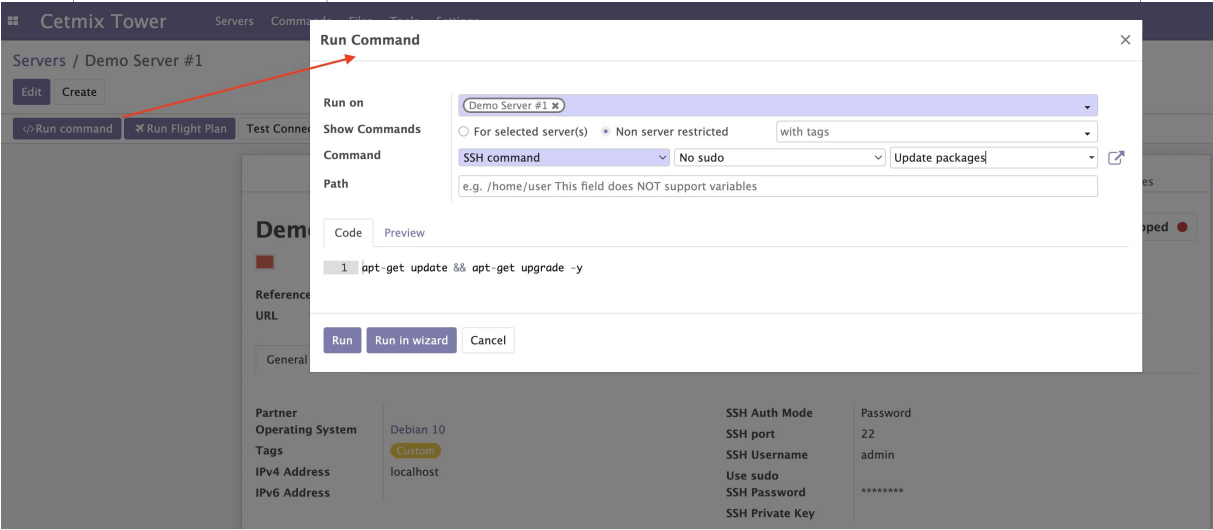


Figura 5.1 — Diálogo Run Command con pestaña Preview del código renderizado.

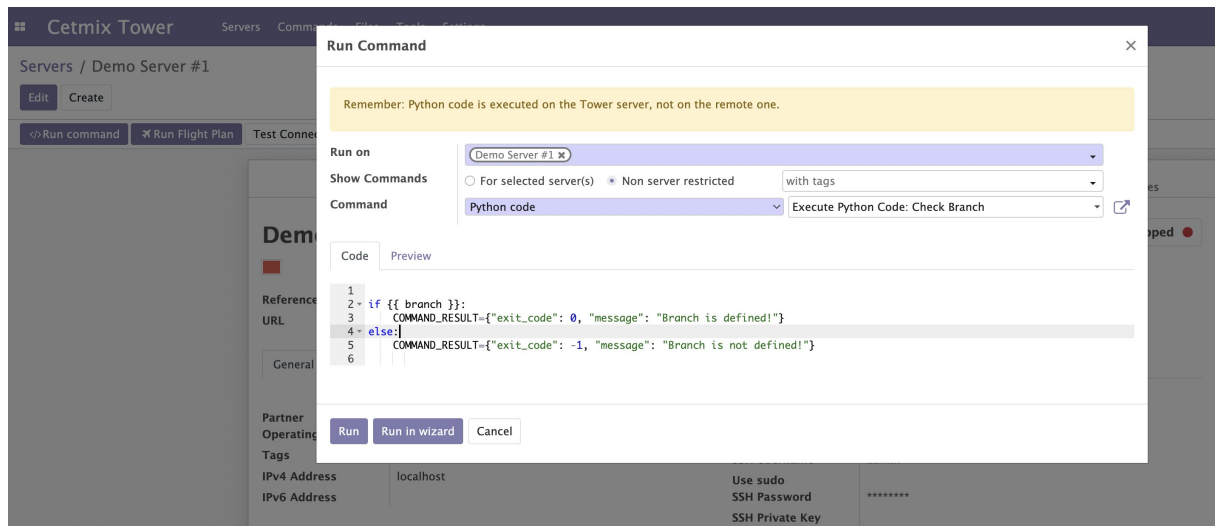


Figura 5.2 — Comando Python en el diálogo de ejecución.

5.3 Comandos Python

```
result = {"exit_code": 0, "message": "Todo bien"}

# Objetos disponibles:
#   server / custom_values / result / ovh / boto3 / objetos Odoo

# Ejemplo: asignación automática de puerto:
usados = server.jet_ids.mapped("variable_value_ids").filtered(
    lambda v: v.variable_id.reference == "odoo_http_port"
).mapped("value_char")
puerto = 10069
while str(puerto) in usados:
    puerto += 100
custom_values["odoo_http_port"] = str(puerto)
result = {"exit_code": 0, "message": "Puerto: %s" % puerto}
```

6. Archivos y Plantillas

Cetmix Tower						
Servers Commands Files Tools Settings						
Files						
Create						
Files Templates						
Search...						
Filters Group By Favorites						
1-9 / 9						
Rendered Name	Source	File Type	File	Server	Last Sync Date	Template
staging_tower_demo_2.txt	Tower	Text		Demo Server #2		Demo File Template 2
repos.yaml	Tower	Text		Demo Server #1		
server_demo_logs.txt	Server	Text		Demo Server #2		Demo File Template 4
server_demo_logs.txt	Server	Text		Demo Server #2		Demo File Template 4
server_demo_logs.txt	Server	Text		Demo Server #1		Demo File Template 4
server_demo.txt	Server	Text		Demo Server #2		
tower_demo_1.txt	Tower	Text		Demo Server #2		Demo File Template 1
tower_demo_3.txt	Tower	Text		Demo Server #2		Demo File Template 3
tower_demo_without_template_staging.txt	Tower	Text		Demo Server #2		

Figura 6.1 — Listado de archivos con columna Source y Rendered Name.

Cetmix Tower						
Servers Commands Files Tools Settings						
Servers / Demo Server #2 / Files / server_demo.txt						
Edit Create						
Pull from Server						
server_demo.txt						
Source	Server	File Type	Text	Auto Sync		
Template		Server	Demo Server #2	Last Sync Date		
Directory on server		Full Server Path	/opt/demo_server_2/cetmix-tower	Variables		
			/opt/demo_server_2/cetmix-tower/server_demo.txt			
Code						

Figura 6.2 — Ficha de un archivo con Source = Server y botón Pull from Server.

Campos de una plantilla (Files > Templates > Nuevo):

Campo	Detalle
File name	Nombre sin ruta. Admite vars: {{ instance_name }}.conf
Directory on server	Ruta destino. Admite vars: {{ base_dir }}/{{ instance_name }}
File content	Cuerpo con variables y secretos.
Source	Tower = verdad en Odoo (se empuja). Server = verdad en server (se lee).
Auto Sync	Descarga periódica (solo Source=Server).

Las 3 plantillas de la instancia Odoo:

Plantilla 1 — docker-compose.yml | Directory: /opt/efactusv/{{ instance_name }} | Source: Tower

```
services:
  odoo:
    container_name: odoo-{{ instance_name }}
    image: {{ odoo_image }}
    restart: always
    depends_on: [db]
    networks: [net_{{ instance_name }}]
    ports:
      - "127.0.0.1:{{ odoo_http_port }}:8069"
      - "127.0.0.1:{{ odoo_gevent_port }}:8072"
    volumes:
      - odoo-web-data:/var/lib/odoo
      - ./config:/etc/odoo
    environment:
      HOST: db
      USER: odoo
      PASSWORD: #!cxtower.secret.db_password!#
      PORT: 5432
    entrypoint: "/usr/bin/odoo -c /etc/odoo/odoo.conf"

db:
  image: postgres:16
  container_name: db-{{ instance_name }}
  restart: always
```

```

environment:
  POSTGRES_USER: odoo
  POSTGRES_PASSWORD: #!cxtower.secret.db_password!#
  POSTGRES_DB: postgres
networks: [net_{{ instance_name }}]
volumes: [odoo-db-data:/var/lib/postgresql/data]

volumes:
  odoo-db-data:
  odoo-web-data:

networks:
  net_{{ instance_name }}:
    driver: bridge

```

127.0.0.1 es obligatorio: sin él Docker publica en todas las interfaces — cualquiera se salta nginx+SSL.

Plantilla 2 — odoo.conf | Directory: /opt/efactusv/{{ instance_name }}/config

```

[options]
addons_path = /mnt/extra-addons,/mnt/efactusv,/mnt/contract,/mnt/knowledge,/mnt/dms
admin_passwd = #!cxtower.secret.admin_passwd!#
db_host = db
db_user = odoo
db_password = #!cxtower.secret.db_password!#
db_name = {{ instance_name }}
dbfilter = ^{{ instance_name }}$
list_db = False
proxy_mode = True
workers = {{ odoo_workers }}
max_cron_threads = 1
gevent-port = 8072
limit_time_cpu = 600
limit_time_real = 1200
transient-age-limit = 1.0
log_level = info

```

Las 3 líneas de aislamiento: dbfilter + list_db=False + proxy_mode=True.

Plantilla 3 — vhost nginx | Directory: /etc/nginx/sites-enabled | File: {{ instance_name }}.conf

```

upstream odoo_{{ instance_name }} {
    server 127.0.0.1:{{ odoo_http_port }}; }
upstream gevent_{{ instance_name }} {
    server 127.0.0.1:{{ odoo_gevent_port }}; }

server {
    listen 80;
    server_name {{ instance_domain }};
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://{{ instance_domain }}$request_uri; }
}

server {
    listen 443 ssl;
    server_name {{ instance_domain }};
    ssl_certificate /etc/letsencrypt/live/{{ instance_domain }}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/{{ instance_domain }}/privkey.pem;
    client_max_body_size 512m;
    proxy_read_timeout 720s;
    location / { proxy_pass http://odoo_{{ instance_name }}; }
    location /websocket {
        proxy_pass http://gevent_{{ instance_name }};
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```

7. Flight Plans

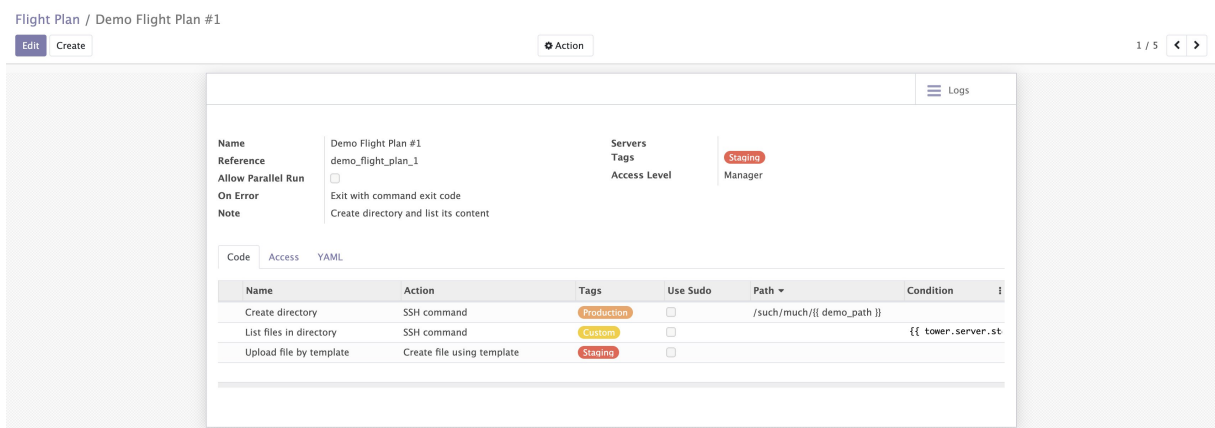


Figura 7.1 — Ficha de un flight plan. Pestaña Code con las líneas del plan.

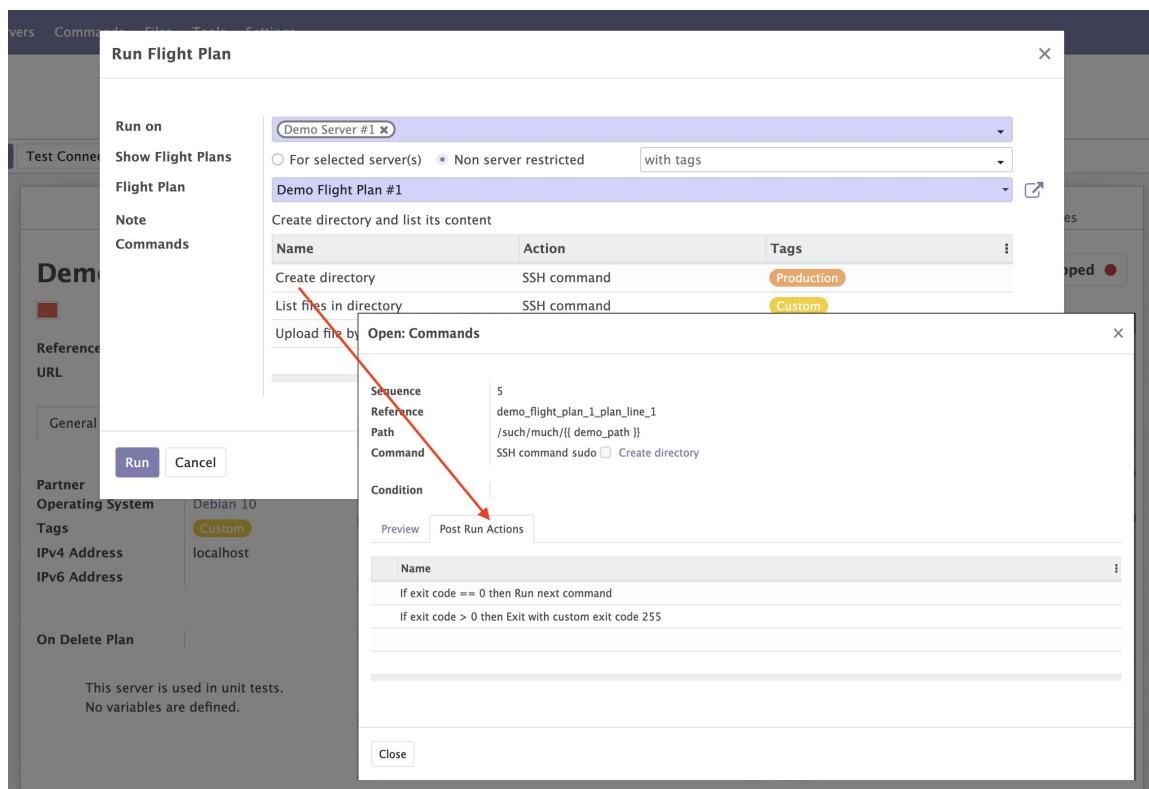


Figura 7.2 — Pestaña Post Run Actions de una línea con el patrón fail-fast.

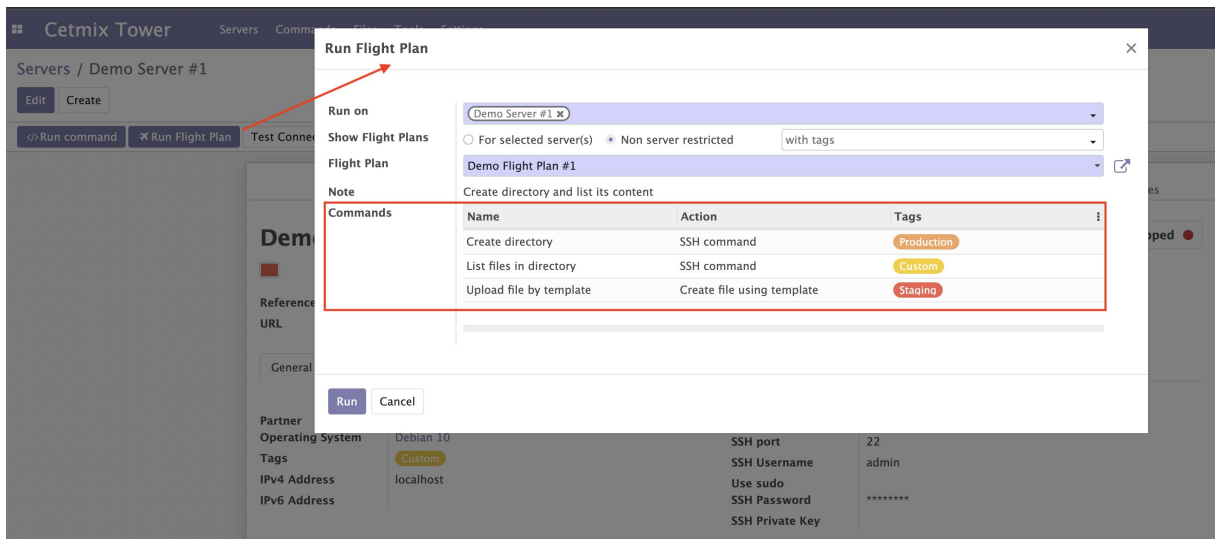


Figura 7.3 — Diálogo Run Flight Plan con lista de comandos a ejecutar.

Estructura de una línea del plan:

Campo	Detalle
Sequence	Orden. Dejar huecos (10, 20, 30...) para insertar.
Command	El comando.
Path	Sobrescribe directorio. Admite variables.
Condition	Si no se cumple → línea omitida (-205).
Post Run Actions	exit_code cond value → Run next / Exit with code / Exit with custom.

Plan maestro — efactusv_instance_install:

Seq	Comando	Acción	Contenido
10	Crear estructura	SSH	mkdir -p {{ base_dir }}/{{ instance_name }}/config
20	Subir docker-compose file	File	Plantilla compose, Overwrite
30	Subir odoo.conf	File	Plantilla odoo.conf, Overwrite
40	Login registry	SSH	docker login ghcr.io con ghcr_token
50	Levantar pila	SSH	cd {{ base_dir }}/{{ instance_name }} && docker compose pull && up -d
60	DNS	Plan	Subplan efactusv_instance_dns (registro A + propagación)
70	SSL + vhost	Plan	Subplan efactusv_instance_ssl (certbot + vhost HTTPS)
80	Esperar PG	SSH	Sondeo pg_isready 60s (ver abajo)
90	Inicializar BD	SSH	odoo -d {{ instance_name }} -i l10n_sv_efactusv --without-demo=all --stop-after-init
99	Registrar URL	Python	Guarda https://{{ instance_domain }} en jet.url

Línea 80:

```
for i in $(seq 1 60); do
    if docker exec db-{{ instance_name }} pg_isready -U odoo -q; then
        echo "PostgreSQL listo tras ${i}s"; exit 0
    fi; sleep 1
done
echo "Timeout"; exit 1
```

Otros planes:

Plan	Contenido
efactusv_instance_stop / _start	docker compose stop / start
efactusv_instance_uninstall	pg_dump → compose down -v → rm vhost + reload → rm -rf dir → borrar DNS
efactusv_server_prepare	Verificar Docker/nginx → mkdir base_dir → mkdir certbot → crear red Docker compartida

Códigos de salida: -201/-301=ejecución en curso, -206=timeout, -207=SO incompatible, -501 a -508=errores de jet.

8. Jets — El Motor SaaS

1 jet = 1 instancia Odoo de cliente. Tiene estados, acciones, ciclo de vida y puede clonarse.

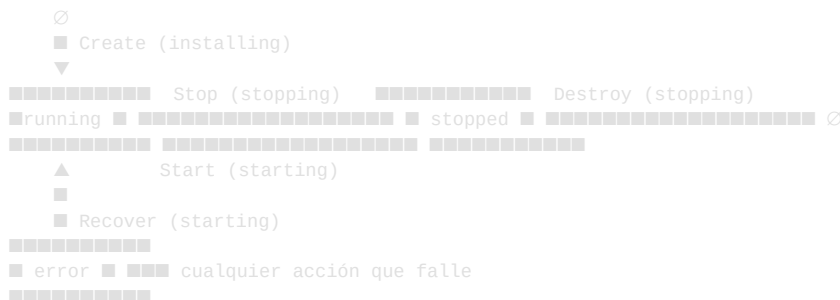
8.1 Estados (Settings > Jets > States)

Referencia	Tipo	Color	Significado
running	Estable	Verde	En marcha.
stopped	Estable	Gris	Contenedores parados, datos intactos.
starting	Transitorio	Azul	Arrancando.
stopping	Transitorio	Azul	Deteniéndose.
installing	Transitorio	Amarillo	Aprovisionamiento inicial.
error	Error	Rojo	Transición falló.

8.2 Acciones (máquina de estados)

Campo	Significado
From State (vacío)	Vacío = acción de creación (desde la nada).
Transit State (obligatorio)	Estado mientras el plan corre. Bloquea acciones concurrentes.
To State (vacío)	Vacío = acción de destrucción (a la nada).
Error State	Adonde va si exit_code != 0.
Flight Plan	El plan que ejecuta la transición.

Ciclo: **estado actual** → **Transit (jet 'ocupado')** → **ejecuta Plan** → **si 0: To State, si no: Error State.**



Tower calcula caminos automáticamente (BFS). 'running → ∅' ejecuta Stop + Destroy.

Acciones de efactusv_odoo_instance:

Acción	From	Transit	To	Error	Flight Plan
Create	∅	installing	running	error	efactusv_instance_install
Stop	running	stopping	stopped	error	efactusv_instance_stop
Start	stopped	starting	running	error	efactusv_instance_start
Recover	error	starting	running	error	efactusv_instance_start
Destroy	stopped	stopping	∅	error	efactusv_instance_uninstall

8.3 Jet Template (Jets > Templates)

Campo	Función
Installation Flight Plan	Plan que corre una vez por servidor al instalar la plantilla.
States / Actions	Máquina de estados.
Files	Plantillas de archivo que se materializan en cada jet.
Variables	Valores por defecto para los jets.
Limit per server	Máx. jets por servidor. Control de densidad.

Campo	Función
Scheduled Tasks	Tareas heredadas (backup diario).

8.4 Crear y operar jets

UI: Jets > Launch New Jet → plantilla, servidor, nombre, variables.

Desde código:

```
jet = jet_template.create_jet(
    server, name="cliente1", partner_id=partner.id,
    variable_values={
        "instance_name": "cliente1",
        "instance_domain": "cliente1.efactusv.com",
        "odoo_http_port": "10069",
        "odoo_gevent_port": "10072",
        "odoo_workers": "2",
    },
)

# API de jets:
jet.bring_to_state("stopped") # respeta permisos
jet.run_command(comando)
jet.run_flight_plan(plan)
jet.get_variable_value("instance_domain")
```

Limit per server alcanzado: create_jet devuelve False (no excepción).

8.5 Asignación automática de puertos

```
jet = env["cx.tower.jet"].browse(jet_id)
servidor = jet.server_id
usados = set()
for otro in servidor.jet_ids.filtered(lambda j: j.id != jet.id):
    valor = otro.get_variable_value("odoo_http_port")
    if valor: usados.add(int(valor))

http = 10069
while http in usados:
    http += 100

# Crear variable values para odoo_http_port y odoo_gevent_port (http+3)
result = {"exit_code": 0, "message": "Puertos: %s / %s" % (http, http+3)}
```

Esquema: 10069, 10169, 10269... http+3 para websocket (mismo desfase 8069→8072).

9. Dominio y SSL

9.1 Subplan DNS (efactusv_instance_dns)

Requiere secretos OVH (ovh_application_key, _secret, consumer_key) y variable ovh_endpoint.

Comando Python (Crear registro A en OVH): comprueba si existe → PUT para actualizar o POST para crear → POST /domain/zone/refresh. **Idempotente**.

Luego comando SSH: bucle getent hosts cada 10s (max 30 intentos) para esperar propagación DNS.

9.2 Subplan SSL (efactusv_instance_ssl)

Seq	Paso	Contenido
10	Vhost HTTP-only	Bloque listen 80 con location /.well-known/acme-challenge/
20	Recargar nginx	sudo nginx -t && sudo systemctl reload nginx
30	Emitir certificado	certbot certonly --webroot -w /var/www/certbot -d {{ instance_domain }} --non-interactive --agree-tos -m {{ admin_email }}
40	Vhost HTTPS	Subir plantilla completa con bloques 80 y 443
50	Recargar nginx	sudo nginx -t && sudo systemctl reload nginx

--keep-until-expiring es crítico: idempotente. Evita chocar con límites LE (5 fallos/h, 50 cert/sem).

9.3 Dominio propio del cliente

Cliente crea CNAME: facturacion.suempresa.com → el tornillo.efactusv.com

Plan efactusv_instance_custom_domain: verificar CNAME → vhost adicional → certbot --expand -d dominio1 -d dominio2 → fijar web.base.url + web.base.url.freeze.

web.base.url.freeze = True evita que Odoo reescriba la URL y los correos salgan con dominio equivocado.

10. Paso a Paso — Crear una Instancia

Checklist previo:

- Servidor producción con Test Connection verde
- Secretos: db_password, admin_passwd, ghcr_token (+ OVH si DNS automático)
- Variables: instance_name, instance_domain, odoo_image, odoo_http_port, odoo_gevent_port, odoo_workers, base_dir, admin_email
- 3 plantillas de archivo: docker-compose.yml, odoo.conf, vhost nginx
- Imagen Docker efactusv publicada en registry

Paso 1 — Crear los 6 estados en Settings > Jets > States: running, stopped, starting, stopping, installing, error.

Paso 2 — Crear los flight plans en Commands > Flight Plans:

- efactusv_instance_install (aprovisionamiento completo)
- efactusv_instance_stop / _start
- efactusv_instance_uninstall
- efactusv_server_prepare
- efactusv_instance_dns / _ssl

Paso 3 — Crear Jet Template 'Instancia Odoo efactusv':

Campo	Valor
Reference	efactusv_odoo_instance
Installation Flight Plan	efactusv_server_prepare
Limit per server	10 (ajustar según VPS)
Variables (defecto)	base_dir, odoo_image, odoo_workers=2, admin_email
Variables (requeridas)	instance_name, instance_domain, odoo_http_port, odoo_gevent_port
Files	Las 3 plantillas de archivo
Actions	Create, Stop, Start, Recover, Destroy

Paso 4 — Instalar plantilla en servidor: abrir template → Install on servers → elegir VPS. Verificar Logs > Jet Template Installations = Done.

Paso 5 — Lanzar instancia:

Campo	Ejemplo
Jet Template / Server	Instancia Odoo efactusv / PROD-SV-01
Name	Ferretería El Tornillo
instance_name	eltornillo
instance_domain	eltornillo.efactusv.com
odoo_http_port / gevent_port	10069 / 10072
odoo_workers	2

→ Disparar **Create** → jet en installing → sigue en Flight Plan Log → al finalizar: jet running (verde), <https://eltornillo.efactusv.com> con login Odoo.

Backup diario (Scheduled Task, Action=Command, Interval=1 Days, Jet Templates=efactusv_odoo_instance):

```
set -e
mkdir -p {{ base_dir }}/backups
docker exec db-{{ instance_name }} pg_dump -U odoo -Fc {{ instance_name }} \
  > {{ base_dir }}/backups/{{ instance_name }}-{{ tower.tools.today_underscore }}.dump
# Backup filestore:
docker run --rm -v {{ base_dir }}/{{ instance_name }}_odoo-web-data:/data:ro \
```


11. Contratos (efactusv_tower_contract)

- Instance Template + Instance Server en el contrato → botón Create Instance
- Al terminar contrato → instancia se detiene a stopped (state on contract termination)
- Al cancelar terminación → vuelve a running
- Fallos de infraestructura NO bloquean administración: se registra en log + chatter.

Flujo de alta:

1. Comercial cierra venta
2. Contrato en Odoo (cliente, plan, facturación)
3. Instance Template + Instance Server en el contrato
4. Botón "Create Instance" → crea jet enlazado
5. Rellenar variables del cliente
6. Disparar "Create" → aprovisionamiento automático
7. Entregar credenciales; facturación activa

Deliberadamente NO destruye al terminar: solo detiene. Destrucción manual tras 30–90 días de retención.

12. Plantillas de Servidor y Git

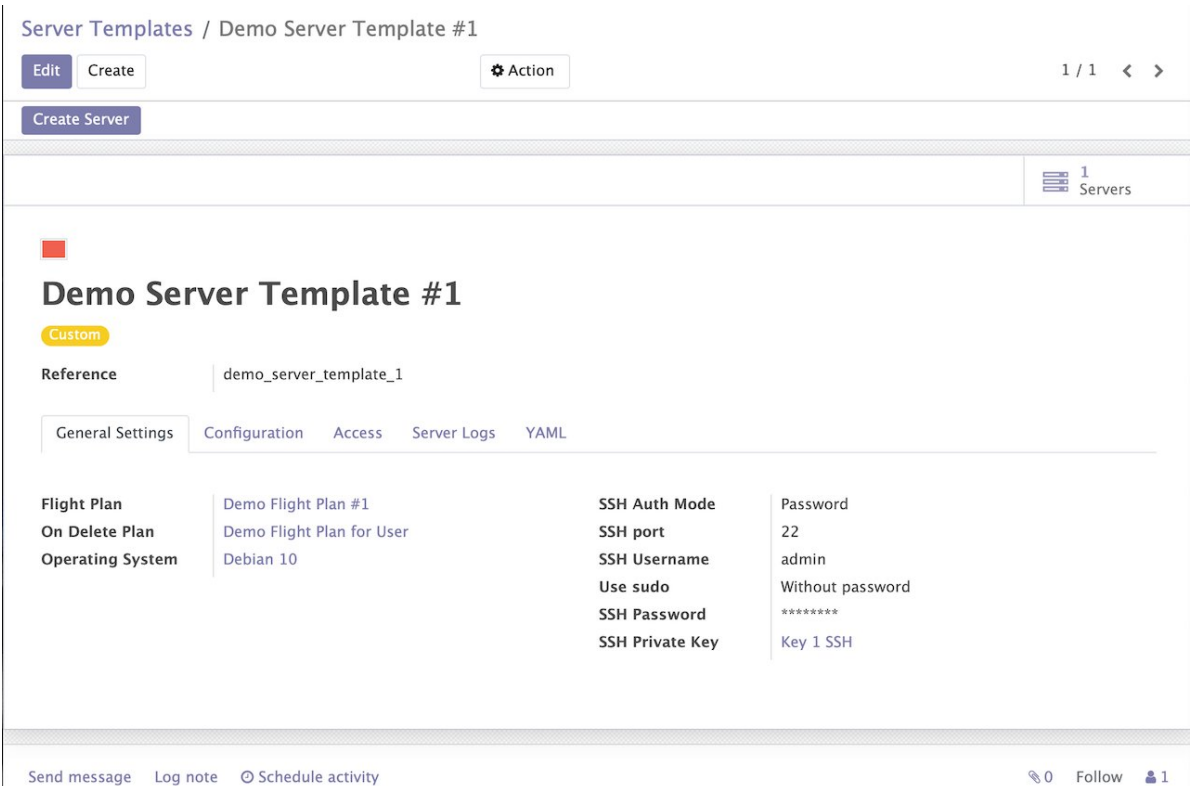


Figura 8.1 — Plantilla de servidor con flight plans del ciclo de vida y parámetros de conexión.

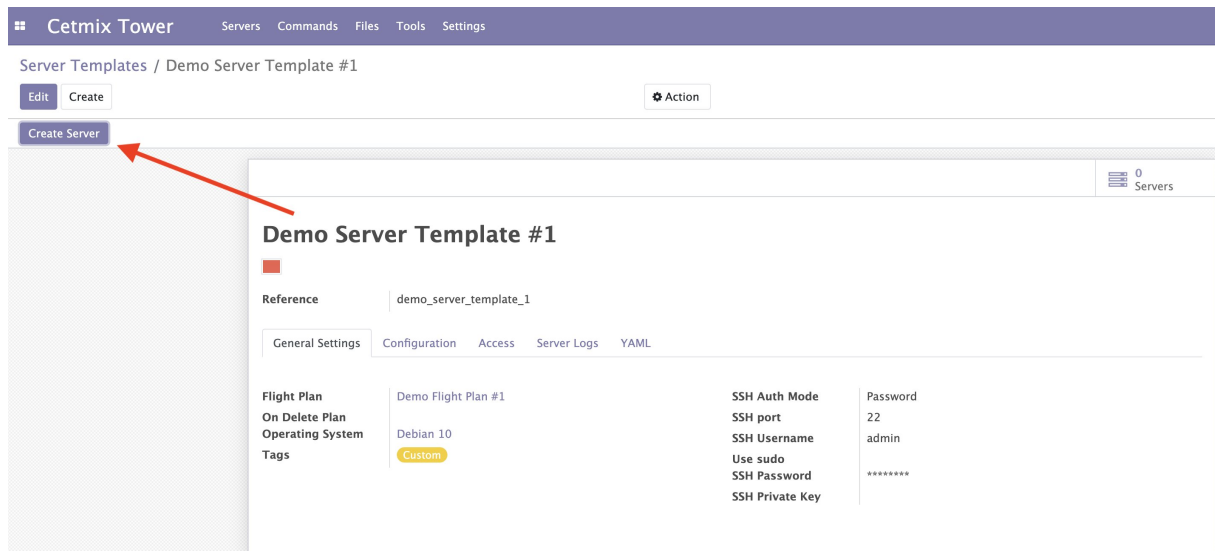


Figura 8.2 — Asistente Create Server desde plantilla. Solo pide nombre, IP y variables requeridas.

create_server_from_template:

```
server = env["cx.tower.server.template"].create_server_from_template(
    "efactusv_prod_vps", "PROD-SV-04",
    partner=partner, ipv4="203.0.113.42", ssh_key="ssh_prod",
    configuration_variables={"base_dir": "/opt/efactusv", "odoo_workers": "4"},
)
```

Proyectos Git (cetmix_tower_git): modela repos, remotos y fuentes. Genera repos.yaml.

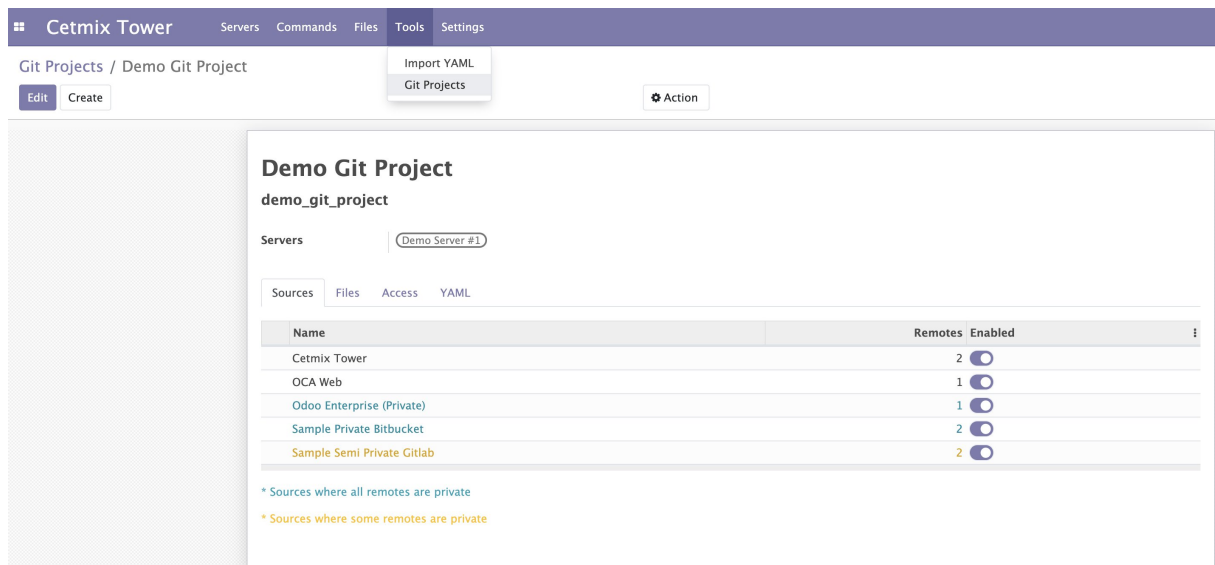


Figura 13.1 — Ficha de un proyecto Git mostrando fuentes y remotos.

Usa imágenes inmutables en producción (código hornado en CI, desplegar = cambiar etiqueta). Reserva Git en servidor para staging.

13. YAML y Versionado

cetmix_tower_yaml exporta/importa: comandos, planes, plantillas, jet templates, variables...

Secretos no viajan: solo la referencia. Al importar, crear secreto con misma referencia en destino.

Flujo: DEV → exportar YAML → Git → PR review → importar en PROD.

Referencias = clave de emparejamiento. No cambiarlas tras exportar.

14. Operación y Diagnóstico

Diario: jets error, Flight Plan Log fallos 24h, backups (Last call), queue jobs fallidos.

Semanal: espacio disco, certificados < 20 días, densidad jets/servidor, probar restore.

Mensual: exportar YAML, revisar accesos Root, jets sin contrato activo.

Diagnóstico rápido:

Síntoma	Causa	Solución
-206	Timeout	Subir Command Timeout o partir comando
-207	SO incompatible	Operating Systems del comando
Jet atascado en transit	Plan falló	Llevar a error → Recover (ver abajo)
502 Bad Gateway	Contenedor caído/puerto mal	docker ps + docker logs
Selector BD visible	dbfilter/list_db	Revisar odoo.conf + reiniciar
No path found	Falta acción	Revisar grafo transiciones
create_jet → False	Limit per server	Otro server o subir límite

Recuperar jet atascado:

```
jet = env["cx.tower.jet"].browse(ID)
estado = env["cx.tower.jet.state"].search([("reference", "=", "error")], limit=1)
jet.sudo().write({"state_id": estado.id, "target_state_id": False})
# Luego: acción Recover → running
```

15. Referencia Rápida

Sintaxis:

Sintaxis	Significado
{{ variable }}	Variable usuario, cascada jet→server→global. Visible en logs.
{{ tower.server.ipv4 }}	Variable sistema, solo lectura.
#!cxtower.secret.x!#	Secreto. Enmascarado en logs.
result = {"exit_code": 0, "message": "..."}\$	Salida comando Python.
custom_values["k"] = v	Pasar datos entre comandos del mismo plan.

API Python esencial:

```
# Servidores
env["cx.tower.server.template"].create_server_from_template(
    ref_plantilla, nombre, ipv4=..., ssh_key=..., configuration_variables={...})
server.test_ssh_connection()
server.run_command(comando) / server.run_flight_plan(plan)

# Jets
jet_template.create_jet(server, name=..., partner_id=..., variable_values={...})
jet_template.install_on_servers(servers)
jet.bring_to_state("stopped") # respeta permisos
jet.get_variable_value("instance_domain")

# Archivos
file_template.create_file(server=server, if_file_exists="skip")
file.upload() / file.download() / file.delete()

# Contratos
contrato.action_create_tower_jet()
contrato.tower_jet_ids
```

Menús principales:

Ruta	Acceso	Contenido
Servers > Servers / Templates	User/Mgr	Servidores y plantillas
Jets > Jets / Templates / Launch	User/Mgr	Instancias y plantillas
Logs > Command / Flight Plan Log	User	Historial ejecuciones
Commands > Commands / Flight Plans	User	Catálogo
Files > Files / Templates	User	Archivos y plantillas
Settings > Keys and Secrets	Manager	SSH keys y secretos
Settings > Jets > States	Manager	Catálogo estados
Settings > Automation > Scheduled Tasks	Manager	Tareas periódicas
Settings > Automation > Webhooks	Root	Endpoints entrantes